كتيّب بحثي شامل

مكتبة os في بايثون

دراسة نظرية وتطبيقية في سياق اختبار الاختراق

فهم طبقة نظام التشغيل • قراءة الكود المصدري • الاستخدام الآمن

إصدار منسق — 2026

فهرس المحتويات

1 الجزء الأول: الإطار النظري والتأسيسي 1
2 الجزء الثاني: الكود المصدري للمكتبة 1
3 الجزء الثالث: طريقة الاستخدام — دليل تطبيقي أمني 1
4 الجزء الأول: الإطار النظري والتأسيسي 1
4.1 مقدمة عامة وتمهيد 1
4.2 تعريف المكتبة ومكانتها في منظومة بايثون القياسية 1
4.3 1 التعريف الاصطلاحي 1
4.4 2 موقعها في تصنيف المكتبات 1
4.5 3 خصائص المكتبة الجوهرية 2
4.6 الفلسفة التصميمية لمكتبة os 2
4.7 1 المبدأ الأول: التجريد لا الإخفاء 2
4.8 2 المبدأ الثاني: التوافقية الرجعية (Backward Compatibility) 2
4.9 3 المبدأ الثالث: البساطة قبل الاكتمال 2
4.10 4 المبدأ الرابع: الاعتماد على النواة 3
4.11 آلية العمل الداخلي (مبسطة) 3
4.12 الوحدات الفرعية والمساندة المرتبطة بـ os 3
4.13 1 الوحدات الفرعية المباشرة 3
4.14 2 الوحدات المرتبطة (المستقلة اسميًا ولكن تعتمد عليها) 4
4.15 مفهوم قابلية النقل (Portability) وأثره الأمني 4
4.16 1 تعريف قابلية النقل 4
4.17 2 آليات تحقيقها في os 4
4.18 3 الآثار الأمنية لقابلية النقل 4
4.19 الأهمية الاستراتيجية للمكتبة في الأمن السيبراني 5
4.20 1 في مرحلة الاستطلاع (Reconnaissance) 5
4.21 2 في مرحلة الاستغلال (Exploitation) 5
4.22 3 في مرحلة ما بعد الاستغلال (Post-Exploitation) 5
4.23 4 في التحليل الجنائي (Forensics) 5
4.24 مقارنة تحليلية مع مكتبات مشابهة 5
4.25 القيود والمخاطر الأمنية النظرية 5
4.26 1 قيود معمارية 5
4.27 2 مخاطر نظرية شائعة 6
4.28 3 حدود المكتبة 6
4.29 أفضل الممارسات النظرية 6
4.30 خلاصة الجزء الأول 6
5 الجزء الثاني: الكود المصدري لمكتبة os — تحليل معمّق 7
5.1 تمهيد 7
5.2 الهيكلية العامة: أين يعيش كود os؟ 7
5.3 تحليل Modules/posixmodule.c — قلب المكتبة النابض 7
5.4 1 طبيعة الملف ومزدوجة المنصة 7
5.5 2 آلية تعريف الدوال: Argument Clinic 8
5.6 3 مثال تحليلي: os.scandir 8
5.7 4 مثال تحليلي: os.remove و os.urandom 8
5.8 تحليل Lib/os.py — الواجهة والمنطق المساعد 9
5.9 1 الاستيراد الشرطي: كيف تختار os وحدتها الأساسية؟ 9

5.10 2 os.walk: مثال على منطق بايثون الخالص 9
5.11 3 os.environ: ليس قاموسًا عاديًا 10
5.12 نقاط الضعف التاريخية والقضايا الأمنية الموثقة 10
5.13 1 CVE-2014-2667: سباق في os.makedirs مع exist_ok=True 10
5.14 2 ثغرات TOCTOU والروابط الرمزية 10
5.15 3 ثغرات Path Traversal 11
5.16 خلاصة الجزء الثاني: ماذا يعني هذا لمختبر الاختراق؟ 11
6 الجزء الثالث: طريقة استخدام مكتبة os — دليل تطبيقي أمني 11
6.1 تمهيد 11
6.2 الاستيراد والتحقق من البيئة 11
6.3 العمل مع الدلائل (Directories) 12
6.4 1 الدليل الحالي وتغييره 12
6.5 2 سرد محتويات الدليل: os.listdir مقابل os.scandir 13
6.6 3 إنشاء وحذف الدلائل 13
6.7 4 اجتياز شجرة الدلائل: os.walk 14
6.8 العمل مع الملفات 14
6.9 1 العمليات الأساسية 14
6.10 2 فحص الصلاحيات: os.stat (Unix-like) 15
6.11 3 الوصول الآمن للملفات: os.open 15
6.12 التعامل مع المسارات: os.path 16
6.13 1 الدوال الأساسية 16
6.14 2 الحماية من Path Traversal 17
6.15 متغيرات البيئة: os.environ 18
6.16 إدارة العمليات 18
6.17 1 معرّف العملية والأب 18
6.18 2 إنشاء عمليات جديدة 19
6.19 3 تنفيذ أوامر النظام: os.system و os.popen 19
6.20 معلومات النظام 20
6.21 الدوال الخاصة بالتحليل الجنائي 21
6.22 1 قراءة الروابط الرمزية 21
6.23 2 الطوابع الزمنية الثلاثة 21
6.24 3 كشف الملفات الحساسة والقابلة للكتابة 21
6.25 الأخطاء الشائعة وكيفية تجنبها 22
6.26 جدول مرجعي شامل لأهم دوال os 22
6.27 خلاصة الجزء الثالث 23
7 مصادر التحقق 25

1 الجزء الأول: الإطار النظري والتأسيسي

1. مقدمة عامة وتمهيد 2. تعريف المكتبة ومكانتها في منظومة بايثون القياسية 3. الفلسفة التصميمية لمكتبة os 4. آلية العمل الداخلي (مبسطة) 5. الوحدات الفرعية والمساندة المرتبطة بـ os 6. مفهوم قابلية النقل وأثره الأمني 7. الأهمية الاستراتيجية في الأمن السيبراني 8. مقارنة تحليلية مع مكتبات مشابهة 9. القيود والمخاطر الأمنية النظرية 10. أفضل الممارسات النظرية 11. خلاصة الجزء الأول

2 الجزء الثاني: الكود المصدري للمكتبة

1. الهيكلية العامة: أين يعيش كود os؟ 2. تحليل Modules/posixmodule.c 3. تحليل Lib/os.py 4. نقاط الضعف التاريخية والقضايا الأمنية الموثقة 5. خلاصة الجزء الثاني

3 الجزء الثالث: طريقة الاستخدام — دليل تطبيقي أمني

1. الاستيراد والتحقق من البيئة 2. العمل مع الدلائل 3. العمل مع الملفات 4. التعامل مع المسارات (os.path) 5. متغيرات البيئة (os.environ) 6. إدارة العمليات 7. معلومات النظام 8. الدوال الخاصة بالتحليل الجنائي 9. الأخطاء الشائعة وكيفية تجنبها 10. جدول مرجعي شامل لأهم الدوال 11. خلاصة الجزء الثالث

4 الجزء الأول: الإطار النظري والتأسيسي

4.1 مقدمة عامة وتمهيد

في عالم اختبار الاختراق والأمن السيبراني، لا يُقاس المحترف بكثرة الأدوات التي يستخدمها، بل بمدى فهمه للطبقات الأساسية التي تتحكم في نظام التشغيل. ومن بين كل المكتبات التي تتعامل معها لغة بايثون، تحتل مكتبة os مكانة استثنائية، فهي الجسر الأول والأخير بين الكود الذي يكتبه المحلل الأمني وبين النواة (Kernel) التي تدير الجهاز.

عندما يكتب مختبر اختراق سكريبتًا لجمع المعلومات عن نظام هدف، أو للبحث عن ملفات قابلة للكتابة بهدف تصعيد الصلاحيات، فإنه في الحقيقة يستدعي — بشكل مباشر أو غير مباشر — دوالًا من مكتبة os. حتى المكتبات المتقدمة مثل pathlib، shutil، subprocess، وpty تعتمد في نهاية المطاف على os كطبقة أساسية.

لذلك، فإن الفهم النظري المعمق لهذه المكتبة ليس ترفًا أكاديميًا، بل هو حجر الأساس لبناء عقلية أمنية قادرة على تحليل أي سكريبت، وفهم آثاره على النظام، والتنبؤ بسلوكه قبل تنفيذه.

4.2 تعريف المكتبة ومكانتها في منظومة بايثون القياسية

4.3 1 التعريف الاصطلاحي

مكتبة os هي إحدى وحدات المكتبة القياسية (Standard Library) في بايثون، وتُعرَّف رسميًا في وثائق بايثون الرسمية بأنها: «وحدات متنوعة لواجهات نظام التشغيل» (Miscellaneous operating system interfaces). وتتمثل وظيفتها الجوهرية في توفير طبقة تجريد (Abstraction Layer) تتيح للكود المكتوب بلغة بايثون التفاعل مع خدمات نظام التشغيل بطريقة موحّدة عبر مختلف المنصات.

4.4 2 موقعها في تصنيف المكتبات

يمكن تصنيف مكتبات بايثون إلى أربع طبقات رئيسية، وos تنتمي إلى الطبقة الأكثر تجذرًا:

جدول 1 — طبقات مكتبات بايثون

الطبقة	الوصف	أمثلة
الطبقة الرابعة: تطبيقات	مكتبات عالية المستوى لمهام محددة	requests، django، scapy
الطبقة الثالثة: خدمات	مكتبات وسيطة تعتمد على الطبقات الأدنى	shutil، subprocess، pathlib، tempfile

الطبقة الثانية: واجهات النظام	أغلفة مباشرة لاستدعاءات النظام	os، sys، socket، signal
الطبقة الأولى: النواة	كود C يتصل مباشرة بنواة نظام التشغيل	posixmodule.c، _io، mmap

الملاحظة الجوهرية: مكتبة os تقع في الطبقة الثانية، وهي بذلك تُعتبر الواجهة الرسمية الأولى التي يلتقي عندها مبرمج بايثون بعالم نظام التشغيل. أي مكتبة أعلى منها (مثل shutil) هي في الحقيقة مجرد تغليف مريح لدوال os.

4.5 3 خصائص المكتبة الجوهرية

تتميز os بمجموعة من الخصائص التي تجعلها فريدة:

- مدمجة (Built-in): لا تحتاج إلى تثبيت عبر pip، فهي تأتي مع أي نسخة بايثون افتراضيًا.
- قياسية (Standard): مُعتمدة رسميًا من مؤسسة Python Software Foundation، وتخضع لسياسة الاستقرار والتوافق الرجعي.
- محمولة (Portable): توفّر نفس الواجهة البرمجية على أنظمة مختلفة (لينكس، ويندوز، ماك، BSD).
- منخفضة المستوى نسبيًا (Low-level): تُقرّب المبرمج من تفاصيل نظام التشغيل، بدلًا من إخفائها.
- حساسة أمنيًا (Security-sensitive): نظرًا لقربها من نظام الملفات والعمليات، فإن أي خطأ في استخدامها قد يؤدي إلى ثغرات حقيقية.

4.6 الفلسفة التصميمية لمكتبة os

4.7 1 المبدأ الأول: التجريد لا الإخفاء

الفلسفة الأساسية لـ os ليست إخفاء تفاصيل نظام التشغيل عن المبرمج، بل تجريدها. بمعنى أن الدالة os.listdir() تعمل بنفس الطريقة على لينكس وويندوز، لكنها لا تخفي حقيقة أن هناك استدعاءات نظام مختلفة تحدث في الخلفية. هذه الفلسفة تختلف جذريًا عن مكتبات مثل pathlib التي تهدف إلى الإخفاء الكامل عبر واجهة كائنية (OOP).

الانعكاس الأمني: هذا يعني أن المحلل الأمني يستطيع — عند الحاجة — التنبؤ بالآثار الحقيقية للسكريبت على النظام، بدلًا من التعامل مع «صندوق أسود».

4.8 2 المبدأ الثاني: التوافقية الرجعية (Backward Compatibility)

تلتزم os بسياسة صارمة تحظر إزالة الدوال القديمة أو تغيير سلوكها بشكل جذري. أي دالة موجودة في os اليوم تعمل بشكلها الحالي منذ إصدارات قديمة، وإن أُهملت (Deprecated) فإنها تُحتفظ بها لفترات طويلة.

الانعكاس الأمني: المحلل الأمني يستطيع الاعتماد على سلوك الدوال عبر إصدارات مختلفة، وهذا مهم في تحليل البرمجيات الخبيثة المكتوبة لإصدارات قديمة من بايثون.

4.9 3 المبدأ الثالث: البساطة قبل الاكتمال

دوال os مصممة لتكون بدائية (Primitive) وليست شاملة. دالة os.remove() تحذف ملفًا فقط، ولا تتعامل مع الحالات المعقدة (مثل حذف مجلد غير فارغ — لهذا توجد shutil.rmtree). هذا التصميم يمنح المبرمج سيطرة دقيقة، لكنه يحمّله مسؤولية التحقق من الحالات الحدّية.

الانعكاس الأمني: السيطرة الدقيقة تعني أن أي إهمال في فحص الحالات الحدّية (مثل os.remove على مسار غير موثوق) قد يتحول إلى ثغرة.

4.10 4 المبدأ الرابع: الاعتماد على النواة

لا تحاول os إعادة اختراع وظائف موجودة في نظام التشغيل. عندما تحتاج لحذف ملف، تستدعي unlink() من النواة مباشرة، ولا تنفذ منطقًا خاصًا بها.

الانعكاس الأمني: هذا يعني أن سلوك os يعتمد على سلوك النواة، وأي ثغرة في النواة تنعكس على os، والعكس صحيح: من يفهم استدعاءات النظام يفهم os فورًا.

4.11 آلية العمل الداخلي (مبسطة)

لفهم كيف تعمل os، يكفي تصور ثلاث طبقات فقط تمر عبرها أي دالة:

```
كود بايثون  →  Lib/os.py  →  Modules/posixmodule.c  →  النواة
```

- Lib/os.py (بايثون): الواجهة العامة. تستورد الدوال من الوحدة الأصلية وتُعيد تصديرها. تحتوي على دوال بايثون خالصة مثل os.walk.
- Modules/posixmodule.c (C): التنفيذ الفعلي. يحوّل وسائط بايثون إلى استدعاءات نظام (System Calls).
- النواة: تنفذ العملية (فتح، حذف، قراءة...).

مثال مبسط: os.remove('file.txt')

1. في os.py: تُستدعى دالة remove المستوردة من الوحدة الأصلية posix (أو nt على ويندوز).
2. في posixmodule.c: تُحوّل المسار إلى نوع C، ثم تُستدعى unlink() من نظام التشغيل.
3. في النواة: تتحقق من الصلاحيات، تحذف الملف، تُرجع النجاح أو الفشل.
4. إما: عودة إلى بايثون: None (نجاح) أو استثناء (FileNotFoundError / PermissionError).

الاستنتاج الأمني: posixmodule.c لا يتحقق من المسار ولا يحمي من الروابط الرمزية. كل التحقق الأمني يجب أن يحدث في كود بايثون قبل الاستدعاء. بكلمات أخرى: المكتبة لا تحميك — أنت من يحمي نفسه.

4.12 الوحدات الفرعية والمساندة المرتبطة بـ os

مكتبة os ليست كتلة واحدة، بل هي عائلة من الوحدات:

4.13 1 الوحدات الفرعية المباشرة

جدول 2 — الوحدات الفرعية المباشرة لـ os

الوحدة	الدور	ملاحظات أمنية
os.path	التعامل مع المسارات (تجميع، تقسيم، تحقق)	نقطة مركزية لثغرات Path Traversal إذا لم تُستخدم os.path.abspath و os.path.realpath

os.environ	قاموس متغيرات البيئة	قد يحتوي على بيانات حساسة (مفاتيح API، كلمات مرور)
os.stat	معلومات الملفات	مفتاح لتحليل الصلاحيات وكشف الملفات الحساسة

4.14 2 الوحدات المرتبطة (المستقلة اسميًا ولكن تعتمد عليها)

جدول 3 — الوحدات المرتبطة بـ os

الوحدة	العلاقة بـ os	الاستخدام الأمني
shutil	مبنية فوق os لعمليات الملفات عالية المستوى	النسخ الاحتياطي، حذف المجلدات الشجرية
subprocess	تستخدم os.exec، os.fork، os.pipe* داخليًا	تنفيذ أوامر بأمان أكبر من os.system
pathlib	تجريد كائني فوق os.path	التلاعب الآمن بالمسارات
tempfile	تعتمد على os.open مع أعلام آمنة	إنشاء ملفات مؤقتة بدون ثغرات
signal	تستخدم os.kill، os.getpid	إدارة العمليات وإيقافها
socket	تعتمد على os.read، os.write	اتصالات الشبكة

4.15 مفهوم قابلية النقل (Portability) وأثره الأمني

4.16 1 تعريف قابلية النقل

قابلية النقل تعني أن الكود المكتوب باستخدام os يمكن تشغيله على أنظمة تشغيل مختلفة دون تعديل، بشرط مراعاة الفروق الموثقة.

4.17 2 آليات تحقيقها في os

- os.name: متغير يُرجع 'posix' أو 'nt'، ليتيح للمبرمج التفريع حسب المنصة.
- os.sep: فاصل المسار (/ أو \).
- os.linesep: فاصل الأسطر (\n أو \r\n).
- os.path: وحدة تُختار تلقائيًا بين posixpath و ntpath حسب المنصة.
- دوال شرطية: بعض الدوال موجودة فقط على منصات معينة (مثل os.fork على لينكس فقط).

4.18 3 الآثار الأمنية لقابلية النقل

هنا يكمن خطر خفي يجب على مختبر الاختراق إدراكه:
المشكلة: كود يعمل بأمان على لينكس قد يكون ثغرة على ويندوز، والعكس صحيح.
أمثلة عملية:

- os.chmod: على أنظمة POSIX تتحكم في بتات الصلاحيات (rwx)، بينما تختلف دلالتها على ويندوز؛ وفي الإصدارات الحديثة يدعم ويندوز قيمة 0o700 بصورة خاصة، أما القيم الأخرى فقد تُتجاهل أو تُفسر وفق المنصة.
- os.fork: متاحة على لينكس لكن غير موجودة على ويندوز، ما قد يدفع مبرمجًا لاستخدام بدائل أقل أمانًا.
- فصل المسارات: os.path.join لا يضمن الأمان وحده؛ يجب التحقق من الاحتواء بعد حل المسار، بينما قد يزيد استخدام + مباشرة من أخطاء فصل المسارات.
- os.access: تتحقق من صلاحية الوصول، لكنها عرضة لثغرات TOCTOU، ويُنصح باستخدام os.open مع الأعلام المناسبة بدلًا منها في السياقات الأمنية.

4.19 الأهمية الاستراتيجية للمكتبة في الأمن السيبراني

4.20 1 في مرحلة الاستطلاع (Reconnaissance)

- os.listdir و os.scandir و os.walk: استكشاف بنية نظام الملفات.
- os.stat: تحليل الصلاحيات والمالكين.
- os.environ: جمع معلومات عن بيئة الهدف.

4.21 2 في مرحلة الاستغلال (Exploitation)

- os.system و os.popen: تنفيذ أوامر (مع خطر Command Injection).
- os.execv: استبدال عملية بعملية أخرى.
- os.setuid و os.setgid: محاولة تصعيد الصلاحيات (على لينكس).

4.22 3 في مرحلة ما بعد الاستغلال (Post-Exploitation)

- os.fork: إنشاء عمليات خلفية (Persistence).
- os.chdir و os.listdir: التنقل في النظام.
- os.remove و os.rename: التلاعب بالملفات وتدمير الأدلة.

4.23 4 في التحليل الجنائي (Forensics)

- os.stat: استخراج الطوابع الزمنية (atime, mtime, ctime).
- os.readlink: تحليل الروابط الرمزية.
- os.getpid و os.getppid: تتبع شجرة العمليات.

4.24 مقارنة تحليلية مع مكتبات مشابهة

جدول 4 — مقارنة os مع مكتبات مشابهة

المعيار	os	pathlib	shutil	subprocess
المستوى	منخفض	عالٍ (كائني)	عالٍ	متوسط
قابلية النقل	عالية	عالية جدًا	عالية	متوسطة (فروق منصة)
الأمان الافتراضي	منخفض (يتطلب حذرًا)	متوسط	متوسط	متوسط
مرونة التحكم	عالية جدًا	متوسطة	متوسطة	عالية
مناسب لتحليل الأمن	ممتاز	جيد	جيد للمهام الجاهزة	ممتاز لتنفيذ الأوامر
الاعتماد عليه في سكريبتات الاختراق	أساسي	مساعد	مساعد	أساسي

الخلاصة: os هي المكتبة الوحيدة التي تجمع بين التحكم الدقيق، الشفافية الأمنية، والاعتماد الأساسي من قبل بقية المكتبات. لا يمكن استبدالها كليًا بأي مكتبة أخرى، بل هي الأساس الذي تُبنى عليه.

4.25 القيود والمخاطر الأمنية النظرية

4.26 1 قيود معمارية

- لا تدير الذاكرة: os لا تتعامل مع إدارة الذاكرة العالية المستوى، بل تترك ذلك لـ mmap و ctypes.

- لا توفر تجريدًا كائنيًا: المسارات نصوص (strings)، وهذا يجعلها عرضة للأخطاء اليدوية.
- لا تفرض سياسات أمنية: لا تمنع حذف ملفات حساسة، ولا تتحقق من المسارات تلقائيًا.

4.27 2 مخاطر نظرية شائعة

1. ثغرات Path Traversal: عند استخدام os.path.join مع مدخلات غير موثوقة دون

```
os.path.abspath.
 2. ثغرات TOCTOU: بين os.access و os.open، أو بين os.path.exists و os.remove.
 3. ثغرات Command Injection: عبر os.system و os.popen.
 4. ثغرات Symlink: عند عدم استخدام follow_symlinks=False.
 5. تسريب المعلومات: عبر رسائل OSError التي قد تكشف مسارات داخلية.
```

4.28 3 حدود المكتبة

- ليست مكتبة شبكات: رغم احتوائها على os.read و os.write، إلا أنها لا توفر تجريدًا للاتصالات.
- ليست مكتبة تشفير: لا تتعامل مع التشفير أو الهاش.
- ليست مكتبة تحليل حزم: ليست بديلًا عن scapy.

4.29 أفضل الممارسات النظرية

1. استخدم os.scandir بدلًا من os.listdir عند الحاجة إلى معلومات الملفات،

```
لتقليل استدعاءات النظام.
 2. جنّب os.system واستبدلها بـ subprocess.run مع shell=False.
 3. استخدم os.open مع أعلام آمنة (O_NOFOLLOW، O_CLOEXEC) في السياقات الحساسة.
 4. تحقق من المسارات باستخدام os.path.realpath و os.path.abspath قبل أي
عملية.
 5. لا تعتمد على os.access في قرارات أمنية، لأنها عرضة لـ TOCTOU.
 6. وثّق الفروق بين المنصات عند كتابة سكريبت يعمل عبر أنظمة مختلفة.
 7. عند عدم الحاجة shutil و pathlib اعتبرها واجهة منخفضة المستوى واستخدم
للتحكم الدقيق.
 8. راقب os.environ (قد يحتوي على أسرار) في السجلات. ولا تُسجل محتواه.
```

4.30 خلاصة الجزء الأول

مكتبة os ليست مجرد مجموعة دوال، بل هي الطبقة التأسيسية التي تربط بايثون بنظام التشغيل. فهمها النظري المعمق يمنح المحلل الأمني ثلاث قدرات جوهرية:

1. القدرة على التنبؤ: معرفة ما سيحدث على النظام عند تنفيذ سكريبت معين. 2. القدرة على التحليل: فهم سلوك أي أداة اختراق مكتوبة ببايثون، لأنها في

```
os. النهاية تستدعي
 3. القدرة على الكتابة الآمنة: تجنب الثغرات الشائعة (Path Traversal، TOCTOU،
Command Injection) من خلال الفهم الصحيح لسلوك الدوال.
```

بهذا الفهم النظري، يصبح الانتقال إلى الجزء الثاني (الكود المصدري) مبنيًا على أساس تقني حقيقي.

5 الجزء الثاني: الكود المصدري لمكتبة os — تحليل معمّق

5.1 تمهيد

في اختبار الاختراق، الاعتماد على «الصندوق الأسود» يعني الاعتماد على افتراضات قد تكون خاطئة. قراءة الكود المصدري لمكتبة os تحوّل المحلل الأمني من مستخدم سطحي إلى مُحلّل قادر على التنبؤ بسلوك الدالة قبل تنفيذها، وفهم أين تكمن الثغرات بالضبط، ولماذا توجد بعض السلوكيات غير البديهية.

5.2 الهيكلية العامة: أين يعيش كود os؟

الكود المصدري لمكتبة os موزّع على ثلاثة مواقع رئيسية داخل مستودع CPython:

جدول 5 — مواقع الكود المصدري لـ os

الموقع	اللغة	الدور
Lib/os.py	Python	الواجهة العامة، الدوال المساعدة، تعريفات المتغيرات، الاستيراد الشرطي
Modules/posixmodule.c	C	التنفيذ الفعلي لمعظم الدوال عبر استدعاءات النظام
Lib/posixpath.py أو Lib/ntpath.py	Python	تنفيذ os.path حسب المنصة

النقطة الجوهرية: os ليست مكتبة بايثون خالصة. الجزء الأكبر من وظائفها الأساسية (مثل os.open، os.remove، os.scandir، os.stat) مكتوب بلغة C، لأن الوصول المباشر لاستدعاءات النظام يتطلب كودًا أصليًا (Native Code). أما Lib/os.py فهي في معظمها طبقة أغلفة (Wrappers) ووظائف مساعدة مبنية فوق ما يوفره posixmodule.c.

5.3 تحليل Modules/posixmodule.c — قلب المكتبة النابض

5.4 1 طبيعة الملف ومزدوجة المنصة

على الرغم من اسمه، فإن posixmodule.c لا يخدم أنظمة POSIX فقط. الملف نفسه ينص في أعلى تعليقاته:

```c
/* POSIX module implementation */
/* This file is also used for Windows NT/MS-Win. In that case the
   module actually calls itself 'nt', not 'posix', and a few functions
   are either unimplemented or implemented differently. */
```

هذا يعني أن نفس الملف المصدر يُترجم إلى وحدتين مختلفتين حسب المنصة: posix على لينكس/ماك/BSD، و nt على ويندوز. هذا التصميم يضمن قابلية النقل عبر واجهة موحدة، لكنه يعني أيضًا أن بعض الدوال قد تكون غائبة أو مختلفة السلوك بين المنصتين.

5.5 2 آلية تعريف الدوال: Argument Clinic

posixmodule.c يستخدم أداة توليد كود تُسمى Argument Clinic. قبل كل دالة نجد تعليقًا خاصًا بالشكل:

```
/*[clinic input]
os.scandir
    path: path_t
    /
Return an iterator of DirEntry objects for given path.
[clinic start generated code]*/
```

هذا التعليق يُستخدم لتوليد كود تحويل الوسائط (من كائنات Python إلى أنواع C) تلقائيًا. الفائدة الأمنية هنا أن التحويل يتم بطريقة موحدة وقابلة للتدقيق، مما يقلل من احتمالية أخطاء تحويل النوع (Type Confusion).

5.6 3 مثال تحليلي: os.scandir

دالة os.scandir هي من أهم الدوال في سياق جمع المعلومات الأمني. تنفيذها في posixmodule.c يمر بالخطوات التالية:

1. استقبال المسار: يتم تحويله إلى نوع path_t (بنية C تحتوي على المسار

```
بصيغتيه الضيقة والعريضة).
 2. اختيار آلية المسح حسب المنصة:
- على أنظمة Unix: تستخدم opendir() و readdir().
- على ويندوز: تستخدم FindFirstFileW و FindNextFileW.
 3. إنشاء كائن ScandirIterator: كائن C وهو مخصص يحتوي على مؤشر الدليل وحالة
التكرار.
 4. إرجاع DirEntry: كل عنصر يُرجع ككائن DirEntry يحتوي على معلومات الملف
(الاسم، النوع، inode) دون الحاجة لاستدعاء stat منفصل لكل ملف.
```

الأهمية الأمنية: هذا التصميم يقلل من عدد استدعاءات النظام، لكنه يعني أن DirEntry قد يحتوي على معلومات قد تكون قديمة (Stale) إذا تغير الملف بين لحظة القراءة ولحظة الاستخدام — وهذا جوهر ثغرات TOCTOU كما سنرى لاحقًا.

5.7 4 مثال تحليلي: os.remove و os.urandom

os.remove (المعروفة أيضًا بـ os.unlink) هي غلاف مباشر حول استدعاء النظام unlink() (أو unlinkat() على لينكس الحديث). لا توجد فيها أي منطق إضافي للتحقق من المسار، ولا أي حماية ضد الروابط الرمزية. كل التحقق الأمني يجب أن يحدث في كود بايثون قبل الاستدعاء.

os.urandom — وهي دالة حساسة أمنيًا لتوليد أرقام عشوائية — تُنفَّذ عبر os_urandom_impl() في posixmodule.c، التي تُفوّض العمل إلى دالة داخلية _PyOS_URandom() المعرّفة في Python/bootstrap_hash.c. هذه الطبقة الإضافية ضرورية لأن os.urandom يجب أن تعمل حتى في مراحل بدء تشغيل بايثون المبكرة جدًا، قبل تهيئة معظم الوحدات.

5.8 تحليل Lib/os.py — الواجهة والمنطق المساعد

5.9 1 الاستيراد الشرطي: كيف تختار os وحدتها الأساسية؟

في بداية Lib/os.py يوجد منطق يشبه ما يلي (مبسّطًا):

```
if 'posix' in _names:
    from posix import *
    import posixpath as path
elif 'nt' in _names:
    from nt import *
    import ntpath as path
```

هذا هو نقطة التحويل الأساسية: عند كتابة import os، يتم استيراد إما posix أو nt حسب المنصة، ثم إعادة تصدير دوالها عبر os. ولهذا فإن os.remove هي في الحقيقة posix.remove أو nt.remove.

5.10 2 os.walk: مثال على منطق بايثون الخالص

os.walk هي إحدى الدوال القليلة في os المكتوبة بالكامل بلغة بايثون، وليس لها مقابل مباشر في posixmodule.c. وهي دالة مولِّدة (Generator) تنفذ اجتيازًا تكراريًا للشجرة؛ ولا ينبغي افتراض أن تنفيذها الداخلي يستدعي نفسها حرفيًا في كل إصدار من CPython. هيكلها المبسّط:

```
def walk(top, topdown=True, onerror=None, followlinks=False):
    try:
        scandir_it = scandir(top)
    except OSError as error:
        if onerror is not None:
            onerror(error)
        return
    with scandir_it:
        while True:
            try:
                entry = next(scandir_it)
            except StopIteration:
                break
            if entry.is_dir():
```

```
        # استدعاء متكرر لـ (walk) لمعالجة المجلدات الفرعية
        ...
```

الملاحظة الأمنية الحاسمة: os.walk تقبل معاملًا يُسمى followlinks، وقيمته

الافتراضية False. إذا تم تعيينه إلى True، فإن الدالة ستتبع الروابط الرمزية، مما قد يؤدي إلى حلقة لا نهائية أو اجتياز خارج الشجرة المقصودة — وهو سلوك خطير في سياقات الفحص الأمني. أيضًا، os.walk تعتمد على os.scandir، فأي ضعف في scandir (مثل عدم الدقة في تحديد نوع الملف) ينتقل إليها.

5.11 3 os.environ: ليس قاموسًا عاديًا

os.environ ليست قاموسًا (dict) عاديًا، بل هي كائن من نوع _Environ يتم إنشاؤه عبر _create_environ_mapping() في Lib/os.py. هذا الكائن يزامن قيمه مع متغيرات البيئة الفعلية للنظام عند القراءة والكتابة. الأثر الأمني: قراءة os.environ قد تكشف بيانات حساسة (مفاتيح API، كلمات مرور، رموز جلسات) موجودة في بيئة العملية. كما أن الكتابة فيه تُغيّر بيئة العملية الحالية وقد تؤثر على العمليات الفرعية التي تُنشأ لاحقًا.

5.12 نقاط الضعف التاريخية والقضايا الأمنية الموثقة

5.13 1 CVE-2014-2667: سباق في os.makedirs مع exist_ok=True

في Lib/os.py، كانت الدالة المساعدة _get_masked_mode (المستخدمة داخل os.makedirs عند تعيين exist_ok=True) تحتوي على حالة سباق (Race Condition). الثغرة موثقة في osv.dev وتنص على أن:

```
"Race condition in the _get_masked_mode function in Lib/os.py in Python
3.2 through 3.5, when exist_ok is set to true and multiple threads are
used, might allow local users to bypass intended file permissions."
```

الشرح: عند إنشاء مجلد بواسطة عدة خيوط (Threads) في نفس الوقت مع exist_ok=True، يمكن لمستخدم محلي استغلال النافذة الزمنية قبل تعيين umask إلى القيمة المتوقعة لتجاوز الصلاحيات المقصودة. هذا مثال كلاسيكي على كيف يمكن لمنطق بايثون الخالص (وليس كود C فقط) أن يحمل ثغرة أمنية.

5.14 2 ثغرات TOCTOU والروابط الرمزية

TOCTOU (Time-of-Check to Time-of-Use) هي فئة الثغرات الأكثر ارتباطًا بمكتبة os. النمط الشائع في كود بايثون:

```
# نمط خطير
if os.path.exists(path):          # CHECK
    os.remove(path)               # USE — قد يكون الملف قد تغير!
```

بين لحظة التحقق (exists) ولحظة الاستخدام (remove)، يمكن للمهاجم استبدال الملف برابط رمزي (Symlink) يشير إلى ملف حساس آخر. هذا النمط موثق في أدلة الأمان كسبب مباشر لثغرات حذف أو اقتطاع ملفات عشوائية.
مثال حقيقي (CVE-2025-68146): ثغرة في مكتبة filelock (التي تعتمد على os.open) سمحت لمهاجم محلي بإنشاء رابط رمزي بين لحظة التحقق من وجود ملف القفل ولحظة فتحه بـ O_TRUNC، مما أدى إلى اقتطاع ملف الضحية. في Unix وmacOS عولجت المشكلة في filelock 3.20.1 بإضافة O_NOFOLLOW إلى os.open؛ أما Windows فاستُخدم فحص مناسب لنقاط إعادة التوجيه (reparse points).
الدرس الجوهري: O_NOFOLLOW يحمي فقط المكوّن الأخير من المسار. إذا كان أي مجلد في المسار (الأساس أو أي سلف) قابلاً للكتابة من قبل المهاجم، فإن استبدال ذلك المجلد برابط رمزي يظل ممكنًا. الحل الكامل هو فتح كل مكوّن على حدة باستخدام os.open(..., dir_fd=...) مع O_NOFOLLOW (دلالات openat).

5.15 3 ثغرات Path Traversal

os.path.join لا تحمي من الهروب خارج الدليل المقصود. إذا مرر المهاجم مسارًا مطلقًا (مثل «/etc/passwd») ، فإن os.path.join(«/app/uploads», «/etc/passwd») ستُرجع «etc/passwd/» وليس المسار المدمج المتوقع. التحقق من «الاحتواء» (Containment) يجب أن يتم عبر os.path.realpath أو Path.resolve() ثم التحقق من أن المسار الناتج لا يزال داخل الدليل الأساسي.

5.16 خلاصة الجزء الثاني: ماذا يعني هذا لمختبر الاختراق؟

جدول 6 — خلاصة الجزء الثاني

النقطة	الدلالة العملية
os ليست بايثون خالصة	فهم استدعاءات النظام (syscalls) ضروري لفهم سلوك os الحقيقي
posixmodule.c مزدوج المنصة	نفس الدالة قد تتصرف بشكل مختلف على لينكس وويندوز
os.scandir يُرجع معلومات قد تكون قديمة	لا تعتمد على DirEntry في قرارات أمنية حساسة دون تحقق لاحق
os.walk مع followlinks=True خطيرة	قد تؤدي لحلقات لا نهائية أو اجتياز خارج النطاق
os.environ كائن حي	قد يحتوي أسرارًا، وقراءته جزء من جمع المعلومات
TOCTOU هي العدو الأول	تحقق-ثم-استخدم = ثغرة محتملة. استخدم O_NOFOLLOW و dir_fd
O_NOFOLLOW ليست كافية وحدها	احمِ كل مكوّن في المسار، وليس المكوّن الأخير فقط

بهذا الفهم للكود المصدري، يصبح الانتقال إلى الجزء الثالث (طريقة الاستخدام) مبنيًا على أساس تقني حقيقي: لن نتعامل مع الدوال كصناديق سوداء، بل كواجهات لسلوكيات نظام محددة، لكل منها مخاطرها وطريقة استخدامها الآمنة.

6 الجزء الثالث: طريقة استخدام مكتبة os — دليل تطبيقي أمني

6.1 تمهيد

هذا الجزء هو الدليل العملي الكامل لاستخدام مكتبة os في سياق الأمن السيبراني واختبار الاختراق. سيتم تغطية كل دالة رئيسية مع مثال كود فعلي يوضح استخدامها، ثم ملاحظة أمنية تحذر من الاستخدام الخاطئ. في نهاية الجزء، ستجد جدولًا مرجعيًا شاملًا لكل الدوال.

6.2 الاستيراد والتحقق من البيئة

قبل أي شيء، يجب التحقق من البيئة التي يعمل عليها السكريبت، لأن سلوك os يتغير حسب المنصة.

```
import os
import sys

print(f"اسم النظام (os.name): {os.name}")           # 'posix' أو 'nt'
print(f"المنصة (sys.platform): {sys.platform}")     # 'linux', 'win32',
'darwin'
print(f"فاصل المسار (os.sep): {os.sep!r}")           # '/' أو '\\'
print(f"فاصل الأسطر (os.linesep): {os.linesep!r}")   # '\n' أو '\r\n'
print(f"فاصل PATH (os.pathsep): {os.pathsep!r}")     # ':' أو ';'
print(f"الدليل الحالي: {os.getcwd()}")
print(f"مسار os.py: {os.__file__}")
```

الملاحظة الأمنية: لا تفترض أبدًا أن السكريبت سيعمل على لينكس فقط. في سكريبتات

اختبار الاختراق، يجب التحقق من os.name قبل استخدام دوال خاصة بمنصة (مثل os.fork).

6.3 العمل مع الدلائل (Directories)

6.4 1 الدليل الحالي وتغييره

```
import os

# الحصول على الدليل الحالي
current = os.getcwd()
print(current)

# تغيير الدليل
os.chdir('/tmp')
print(os.getcwd())   # /tmp
```

الملاحظة الأمنية: os.chdir تغيّر حالة العملية بأكملها، وليس الخيط الحالي فقط. في

سكريبت متعدد الخيوط (Multithreaded)، قد يؤدي هذا إلى سلوك غير متوقع. يُفضل استخدام المسارات المطلقة بدلًا من تغيير الدليل.

6.5 2 سرد محتويات الدليل: os.listdir مقابل os.scandir

```python
import os

# الطريقة القديمة — تُرجع أسماء فقط
for name in os.listdir('/etc'):
    print(name)

# الطريقة الحديثة والآمنة DirEntry — تُرجع كائنات
with os.scandir('/etc') as entries:
    for entry in entries:
        print(f"{entry.name:30s} | is_file={entry.is_file()} | "
              f"is_dir={entry.is_dir()} | is_symlink={entry.is_symlink()}")
```

لماذا os.scandir أفضل أمنيًا؟

- os.listdir تُرجع قائمة أسماء بدون معلومات، مما يضطرك لاستدعاء os.stat لكل ملف (استدعاء نظام إضافي).
- os.scandir تُرجع DirEntry يحتوي على معلومات النوع مخزنة مؤقتًا، وتستطيع تمييز الروابط الرمزية (is_symlink()) قبل فتحها، وهذا حاسم لتجنب ثغرات Symlink.
- تحذير: entry.is_file() و entry.is_dir() تتبع الروابط الرمزية افتراضيًا. للتحقق من النوع الحقيقي دون اتباع الروابط، استخدم entry.is_file(follow_symlinks=False).

6.6 3 إنشاء وحذف الدلائل

```python
import os

# إنشاء مجلد واحد
os.mkdir('/tmp/mydir')

# إنشاء مجلدات متداخلة
os.makedirs('/tmp/a/b/c', exist_ok=True)

# حذف مجلد فارغ
os.rmdir('/tmp/mydir')

# حذف مجلدات متداخلة (يعمل فقط إذا كانت فارغة)
os.removedirs('/tmp/a/b/c')
```

الملاحظة الأمنية: os.makedirs مع exist_ok=True كانت لها ثغرة تاريخية

(CVE-2014-2667) في بيئات متعددة الخيوط. لا تعتمد على فحص منفصل ثم الإنشاء، لأن ذلك قد يعيد نمط TOCTOU. استخدم إصدار Python مدعومًا، وتعامل مع FileExistsError وOSError وفق الحالة.

6.7 4 اجتياز شجرة الدلائل: os.walk

هذه من أهم الدوال في سياق جمع المعلومات (Enumeration).

```
import os

for root, dirs, files in os.walk('/var/www', followlinks=False):
    print(f"[+] الدليل: {root}")
    for d in dirs:
        print(f"    [دليل]  {d}")
    for f in files:
        print(f"    [ملف]   {f}")
```

الملاحظة الأمنية الحاسمة: followlinks=True قد يؤدي إلى حلقة لا نهائية إذا كان

هناك رابط رمزي يشير إلى دليل أعلى في الشجرة. كما أنه قد يؤدي إلى اجتياز مساحات خارج النطاق المقصود (مثل /proc/self/root). دائمًا استخدم followlinks=False ما لم يكن لديك سبب واضح.

6.8 العمل مع الملفات

6.9 1 العمليات الأساسية

```
import os

# إنشاء ملف فارغ (آمن نسبيًا)
with open('/tmp/test.txt', 'w') as f:
    f.write('hello')

# حذف ملف
os.remove('/tmp/test.txt')         # أو os.unlink

# إعادة التسمية / النقل
os.rename('/tmp/old.txt', '/tmp/new.txt')

# التحقق من الوجود
print(os.path.exists('/etc/passwd'))   # True
print(os.path.isfile('/etc/passwd'))   # True
```

```
print(os.path.isdir('/etc'))           # True
print(os.path.islink('/etc/rc.local')) # True إذا كان رابطًا رمزيًا
```

الملاحظة الأمنية على os.rename: على أنظمة POSIX، os.rename تستبدل الملف الهدف إن

كان موجودًا دون تحذير. في سكريبتات الأمن، إذا كنت تنقل ملفًا إلى مسار قد يتحكم فيه المهاجم، فقد يستبدل ملفًا حساسًا. استخدم os.replace (سلوكها مماثل لكنها تعمل عبر المنصات بشكل أكثر اتساقًا).

6.10 2 فحص الصلاحيات: os.stat (Unix-like)

```
import os
import stat

st = os.stat('/etc/passwd')
print(f"الحجم: {st.st_size} بايت")
print(f"المالك (UID): {st.st_uid}")
print(f"المجموعة (GID): {st.st_gid}")
print(f"الصلاحيات (octal): {oct(stat.S_IMODE(st.st_mode))}")
print(f"آخر وصول: {st.st_atime}")
print(f"آخر تعديل: {st.st_mtime}")

# فحص صلاحيات محددة
if st.st_mode & stat.S_IWOTH:
    print("[خطر] الملف قابل للكتابة من قبل أي مستخدم!")
if st.st_mode & stat.S_ISUID:
    print("[تنبيه] الملف يحمل بت SUID – قد يسمح بتصعيد صلاحيات!")
```

الملاحظة الأمنية: البحث عن الملفات ذات بت SUID أو SGID هو خطوة أساسية في تصعيد

الصلاحيات على لينكس. استخدم os.walk + os.stat للبحث عنها في النظام.

6.11 3 الوصول الآمن للملفات: os.open

عندما تحتاج إلى تحكم دقيق في كيفية فتح الملف (خاصة في السياقات الأمنية)، استخدم os.open بدلًا من open المدمجة:

```python
import os
import stat

# فتح ملف للقراءة فقط، مع رفض الروابط الرمزية
try:
    fd = os.open(
        '/etc/passwd',
        os.O_RDONLY | os.O_NOFOLLOW,   # O_NOFOLLOW يرفض الروابط الرمزية
        0o600
    )
    with os.fdopen(fd, 'r') as f:
        data = f.read()
        print(data[:100])
except OSError as e:
    print(f"فشل الفتح: {e}")
```

شرح الأعلام الأمنية:

جدول 7 — الأعلام الأمنية في os.open

العلم	الوظيفة	الفائدة الأمنية
O_NOFOLLOW	يرفض فتح الملف إذا كان رابطًا رمزيًا	يمنع هجمات Symlink
O_CLOEXEC	يغلق الواصف تلقائيًا عند تنفيذ exec	يمنع تسريب الواصفات للعمليات الفرعية
O_EXCL	يفشل إذا كان الملف موجودًا (مع O_CREAT)	يمنع الكتابة فوق ملفات موجودة
O_CREAT	إنشاء الملف إذا لم يكن موجودًا	—
O_TRUNC	يقتطع الملف إلى طول صفر	خطير إذا استُخدم مع مسار غير موثوق

تحذير متقدم: O_NOFOLLOW تحمي آخر مكوّن فقط في المسار. إذا كان مجلد أب في المسار قابلاً للكتابة من قبل المهاجم، فيمكنه استبدال المجلد برابط رمزي. الحل الكامل هو فتح كل مكوّن على حدة باستخدام os.open(..., dir_fd=...) — وهو نمط متقدم يُعرف باسم openat pattern.

6.12 التعامل مع المسارات: os.path

6.13 1 الدوال الأساسية

```python
import os

# دمج المسارات
os.path.join('/etc', 'ssh', 'sshd_config')   # '/etc/ssh/sshd_config'
```

```
# تقسيم المسار
os.path.split('/etc/ssh/sshd_config')   # ('/etc/ssh', 'sshd_config')
os.path.dirname('/etc/ssh/sshd_config') # '/etc/ssh'
os.path.basename('/etc/ssh/sshd_config')# 'sshd_config'

# فصل الامتداد
os.path.splitext('malware.exe')         # ('malware', '.exe')

# الحصول على المسار المطلق (دون حل الروابط)
os.path.abspath('./file.txt')

# الحصول على المسار الحقيقي (يحل الروابط الرمزية و ..)
os.path.realpath('/etc/rc.local')
```

6.14.2 الحماية من Path Traversal

هذا مثال لتقليل كثير من حالات Path Traversal عند التعامل مع مسار إدخال غير موثوق. لا يقدّم وحده حماية كاملة من تغيّر المسار بعد التحقق أو من مجلدات يملك المهاجم صلاحية تعديلها:

```
import os

def safe_join(base_dir: str, user_path: str) -> str | None:
    """يدمج مسارًا بشكل آمن، ويرفض أي محاولة للهروب من base_dir."""
    # 1. الحصول على المسار المطلق الحقيقي للقاعدة
    base = os.path.realpath(base_dir)
    # 2. دمج المسار ثم حل الروابط
    target = os.path.realpath(os.path.join(base, user_path))
    # 3. التحقق من أن الهدف داخل القاعدة
    #    نضيف os.sep لتجنب حالة /appfoo مقابل /app
    if target == base or target.startswith(base + os.sep):
        return target
    return None

# اختبار
print(safe_join('/app/uploads', 'report.pdf'))        # /app/uploads/
report.pdf  (صح)
print(safe_join('/app/uploads', '../../etc/passwd'))  # None  (مرفوض)
print(safe_join('/app/uploads', '/etc/passwd'))       # None  (مرفوض)
```

لماذا يساعد هذا؟

- realpath يحل كل الروابط الرمزية و .. قبل التحقق.
- إضافة os.sep تمنع حالة الخداع الشهيرة: app/uploads/ مقابل app/uploads_evil/.
- os.path.join وحدها لا تكفي — يجب إقرانها بالتحقق من البادئة بعد realpath.

ملاحظة متقدمة: على ويندوز، يجب أن تأخذ في الحسبان أحرف محرك الأقراص (C:\)، والمسارات UNC (\\server\share)، وسلوك realpath الذي يختلف. أيضًا، التحقق من البادئة النصية لا يكفي إذا كان النظام يستخدم روابط رمزية داخل القاعدة نفسها — هنا تحتاج إلى os.open مع dir_fd أو إلى مقارنة inode.

6.15 متغيرات البيئة: os.environ

```
import os

# قراءة متغير
print(os.environ.get('PATH'))
print(os.environ.get('HOME'))

# كتابة متغير (يؤثر على العملية الحالية والعمليات الفرعية)
os.environ['MY_VAR'] = 'value'

# حذف متغير
os.environ.pop('MY_VAR', None)

# سرد كل المتغيرات (قد يكشف أسرارًا!)
for key, value in os.environ.items():
    print(f"{key}={value}")
```

الملاحظة الأمنية: في سكريبتات جمع المعلومات، os.environ قد تحتوي على:

- مفاتيح API (AWS_ACCESS_KEY_ID، GITHUB_TOKEN).
- بيانات اعتماد قواعد البيانات (DATABASE_URL).
- مسارات داخلية حساسة (PYTHONPATH).

لا تُسجل محتوى os.environ في السجلات (Logs) لأن ذلك يُسرّب الأسرار. عند الحاجة، سجّل أسماء المتغيرات فقط دون قيمها.

6.16 إدارة العمليات

6.17 1 معرّف العملية والأب

```
import os
```

```
print(f"PID الحالي: {os.getpid()}")
print(f"PID الأب: {os.getppid()}")
print(f"UID: {os.getuid()}")        # لينكس فقط
print(f"EUID: {os.geteuid()}")      # لينكس فقط — مهم لتصعيد الصلاحيات
```

الملاحظة الأمنية: os.geteuid() (Effective UID) هو المفتاح لمعرفة ما إذا كان

السكريبت يعمل بصلاحيات مرتفعة. في سكريبتات تصعيد الصلاحيات، الفرق بين getuid و geteuid هو أول ما يتم فحصه.

6.18 2 إنشاء عمليات جديدة

```
import os

# لينكس فقط — إنشاء عملية فرعية
pid = os.fork()
if pid == 0:
    # نحن في العملية الابن
    print(f"الابن: PID={os.getpid()}, الأب={os.getppid()}")
    os._exit(0)
else:
    # نحن في العملية الأب
    print(f"الأب: PID={os.getpid()}, الابن={pid}")
    os.waitpid(pid, 0)
```

الملاحظة الأمنية: os.fork غير متاحة على ويندوز. في سكريبتات Persistence، قد

يستخدم المهاجم os.fork لإنشاء عملية خلفية تتجاوز إنهاء العملية الأصلية. راقب os.fork في تحليل البرمجيات الخبيثة.

6.19 3 تنفيذ أوامر النظام: os.system و os.popen

```
import os

# خطر: os.system
os.system('ls -la /tmp')
```

```
# خطر: os.popen
output = os.popen('id').read()
print(output)
```

التحذير الأمني الحاسم: os.system و os.popen تمرران الأمر إلى shell النظام، مما

يجعل السكريبت عرضة لثغرة Command Injection إذا تم تضمين مدخلات غير موثوقة:

```
# كود خطير جدًا
filename = input("اسم الملف: ")          # المستخدم يدخل: "a; rm -rf /"
os.system(f"cat {filename}")              # ينفذ: cat a; rm -rf /

# البديل الآمن
import subprocess
subprocess.run(['cat', filename], shell=False, check=True)
```

القاعدة العملية: تجنّب os.system وos.popeng عندما يكون الأمر أو أي جزء منه قابلًا للتأثر بمدخلات غير موثوقة. استخدم subprocess.run مع shell=False وتمرير الأمر كقائمة؛ فهذا لا يلغي الحاجة إلى التحقق من الوسائط.

6.20 معلومات النظام

```
import os

# عدد أنوية المعالج
print(f"أنوية المعالج: {os.cpu_count()}")

# حجم الكتلة في نظام الملفات
print(f"حجم الكتلة: {os.statvfs('/').f_bsize}")   # لينكس فقط

# المسافة الفاصلة في PATH
print(os.environ.get('PATH', '').split(os.pathsep))

# الملف /dev/null (أو NUL على ويندوز)
print(os.devnull)
```

الملاحظة الأمنية: os.cpu_count() و os.statvfs تُستخدم في بصمة النظام (System)

Fingerprinting) لمعرفة إن كان الهدف جهازًا حقيقيًا أم افتراضيًا (Sandbox).

6.21 الدوال الخاصة بالتحليل الجنائي

6.22 1 قراءة الروابط الرمزية

```
import os

if os.path.islink('/etc/rc.local'):
    target = os.readlink('/etc/rc.local')
    print(f"الرابط يشير إلى: {target}")
```

6.23 2 الطوابع الزمنية الثلاثة

```
import os

st = os.stat('/etc/passwd')
print(f"آخر وصول (atime): {st.st_atime}")
print(f"آخر تعديل (mtime): {st.st_mtime}")
print(f"آخر تغيير للـ inode (ctime): {st.st_ctime}")
```

الملاحظة الجنائية: ctime ليس وقت الإنشاء. إنه وقت آخر تغيير في بيانات inode

(مثل الصلاحيات أو المالك). للتحقق من وقت الإنشاء على لينكس، تحتاج إلى os.stat مع st_birthtime (غير متاح على كل الأنظمة) أو قراءة statx عبر ctypes. على ويندوز، os.stat يُرجع st_ctime كوقت إنشاء فعليًا — وهذا فرق جوهري بين المنصتين.

6.24 3 كشف الملفات الحساسة والقابلة للكتابة

```
import os
import stat

def find_world_writable(root):
    """يبحث عن ملفات قابلة للكتابة من قبل الجميع."""
    findings = []
    for dirpath, dirnames, filenames in os.walk(root, followlinks=False):
        for name in filenames:
```

```
            full = os.path.join(dirpath, name)
            try:
                st = os.lstat(full)   # لا يتبع الروابط lstat
            except OSError:
                continue
            if st.st_mode & stat.S_IWOTH:
                findings.append(full)
    return findings

# مثال
for f in find_world_writable('/tmp'):
    print(f"[!] قابل للكتابة: {f}")
```

الملاحظة الأمنية: استخدم os.lstat بدلًا من os.stat لتحليل الملفات نفسها دون اتباع الروابط الرمزية. os.lstat تُرجع معلومات الرابط نفسه، وليس الملف الذي يشير إليه.

6.25 الأخطاء الشائعة وكيفية تجنبها

جدول 8 — الأخطاء الشائعة وكيفية تجنبها

الخطأ	النتيجة	الحل الصحيح
os.path.exists() ثم os.remove()	ثغرة TOCTOU	استخدم os.open مع O_NOFOLLOW، أو os.unlink مباشرة مع معالجة الخطأ
os.system(user_input)	Command Injection	subprocess.run([…], shell=False)
os.path.join(base, user_input) دون تحقق	Path Traversal	realpath + التحقق من البادئة
os.walk(…, followlinks=True)	حلقة لا نهائية / خروج عن النطاق	followlinks=False
os.listdir ثم os.stat لكل ملف	استدعاءات نظام مضاعفة	os.scandir
os.stat بدل os.lstat	اتباع روابط رمزية عن طريق الخطأ	os.lstat
os.environ في السجلات	تسريب أسرار	سجّل الأسماء فقط
from os import *	استبدال open() المدمجة بـ os.open()	import os فقط

6.26 جدول مرجعي شامل لأهم دوال os

جدول 9 — الجدول المرجعي الشامل لأهم دوال os

الدالة	الوصف	تحذير أمني
os.getcwd()	الدليل الحالي	—

os.chdir(p)	تغيير الدليل	يؤثر على العملية بأكملها
os.listdir(p)	سرد أسماء	لا يميز الأنواع
os.scandir(p)	سرد كائنات DirEntry	الأفضل أمنيًا
os.walk(p)	اجتياز شجري	followlinks=True خطير
os.mkdir(p)	إنشاء مجلد	—
os.makedirs(p)	إنشاء متداخل	CVE تاريخي مع exist_ok
os.rmdir(p)	حذف مجلد فارغ	—
os.remove(p)	حذف ملف	لا حماية من Symlink
os.rename(a,b)	إعادة تسمية	يستبدل الهدف
os.stat(p)	معلومات الملف	يتبع الروابط
os.lstat(p)	معلومات الرابط	لا يتبع الروابط — آمن للتحليل
os.readlink(p)	هدف الرابط	—
os.open(p, flags)	فتح منخفض المستوى	استخدم O_NOFOLLOW
os.close(fd)	إغلاق واصف	—
os.getpid()	PID الحالي	—
os.getppid()	PID الأب	—
os.getuid()	UID الحقيقي	لينكس فقط
os.geteuid()	UID الفعلي	مفتاح تصعيد الصلاحيات
os.fork()	إنشاء عملية	لينكس فقط، غير آمن متعدد الخيوط
os.system(cmd)	تنفيذ أمر	Command Injection
os.popen(cmd)	تنفيذ وقراءة	Command Injection
os.environ	متغيرات البيئة	قد تحتوي أسرارًا
os.cpu_count()	عدد الأنوية	بصمة النظام
os.devnull	مسار /dev/null	—
os.path.join(a,b)	دمج مسارات	لا يحمي من Path Traversal
os.path.abspath(p)	مسار مطلق	لا يحل الروابط
os.path.realpath(p)	مسار حقيقي	يحل الروابط — الأفضل للتحقق
os.path.exists(p)	وجود	TOCTOU
os.path.isfile(p)	ملف؟	يتبع الروابط
os.path.islink(p)	رابط؟	—
os.path.splitext(p)	فصل الامتداد	—

6.27 خلاصة الجزء الثالث

المكتبة os في الاستخدام العملي تنقسم إلى ثلاث فئات:

1. دوال آمنة نسبيًا إذا استُخدمت بشكل صحيح: os.lstat، os.scandir،

```
os.path.realpath، os.readlink.
 2. دوال خطيرة تتطلب حذرًا: os.remove، os.rename، os.open (بدون أعلام)،
os.walk (مع followlinks).
 3. دوال يجب تجنبها في الكود الإنتاجي: os.system، os.popen، os.path.join
(بدون تحقق لاحق).
```

القاعدة العامة: كل دالة في os تفترض أن المتصل قد تحقق من كل شيء. المكتبة لا تحميك — أنت من يحمي نفسه عبر الفهم الصحيح، والأعلام الآمنة، والتحقق من المسارات، وتجنّب أنماط TOCTOU.

خلاصة الكتيّب

هذا الكتيّب يقدّم مسارًا عمليًا لفهم مكتبة os من طبقة الواجهة البرمجية إلى استدعاءات النظام، مع إبراز أن الأمان لا يأتي من المكتبة وحدها، بل من التحقق الصحيح من المسارات والصلاحيات والروابط الرمزية وتجنب أنماط TOCTOU وحقن الأوامر.

7 مصادر التحقق

تمت مراجعة النقاط التقنية والأمنية الحساسة بالرجوع إلى المصادر التالية:

[1] Python Documentation — os module: docs.python.org/3/library/os.html [2] Red Hat — CVE-2014-2667: access.redhat.com/security/cve/cve-2014-2667 [3] Python Security — os.makedirs historical advisory: python-security.readthedocs.io/vuln/os-makedirs-not-thread-safe.html [4] GitHub Advisory — filelock TOCTOU: github.com/advisories/GHSA-w853-jp5j-5j7f